MyEngine

A from-scratch 2D game engine in C# / .NET 10 + OpenGL (Silk.NET)

Full code guide — every file, every line, and every concept behind it

Generated for the MyEngine devlog project
Solution: `MyEngine.slnx` — projects `Engine` (class library) and `Game` (entry point)
Covers: Episodes 1–3 (window, triangle, sprites) and the platformer prototype (acceleration/deceleration, gravity, AABB collision, coyote time, jump buffering)

Contents

1. Project overview & philosophy
2. Solution layout & build system
3. The NuGet packages, and what each one buys you
4. Core concepts primer
    1. The GPU pipeline in one page
    2. VAO, VBO, EBO — what they actually are
    3. Shaders, compiling, linking, uniforms
    4. Textures, texture units, filtering, mipmaps
    5. Coordinate spaces & the projection matrix
    6. The game loop and deltaTime
    7. C# features this codebase leans on
5. Program.cs — the entry point
6. GameWindow.cs — window, input, the game loop
7. Shader.cs — compiling and talking to a GL program
8. Texture.cs — loading a PNG onto the GPU
9. Sprite.cs — a drawable textured quad
10. sprite.vert / sprite.frag — the actual GLSL
11. Aabb.cs — axis-aligned bounding boxes
12. Player.cs — the platformer controller, in depth
13. Game-feel mechanics, worked examples
14. Glossary
15. Current state & what's next

1. Project overview & philosophy

MyEngine is a 2D game engine being built **from scratch** — no existing engine (Unity, Godot, MonoGame) underneath. It talks directly to the graphics card through **OpenGL**, using **Silk.NET** as the thin C# binding layer that exposes OpenGL's C API to .NET. The point of building it this way is educational: every triangle that ends up on screen exists because this codebase explicitly told the GPU to put it there, with no hidden engine machinery doing it automatically.

The architecture is deliberately simple and object-oriented rather than an Entity-Component-System (ECS). If you've used Unity, the mental model to reach for is: `GameWindow` is roughly analogous to Unity's own engine loop (the thing that calls `Update()` and renders every frame), and classes like `Player` are roughly analogous to a `MonoBehaviour` — except here there's no engine underneath doing transform hierarchies, physics, or rendering for you. Every one of those systems is hand-written in this project, and this document explains exactly how.

The codebase is still small enough to read start to finish in under an hour. That's intentional — it's built incrementally, episode by episode, and this guide documents the project as it stands right now: window creation, textured sprite rendering, and a platformer character controller with smooth movement and forgiving jump timing.

2. Solution layout & build system

The repository is a single Visual Studio solution (`MyEngine.slnx`) containing two sibling projects:

```
myengine\
├─ MyEngine.slnx
├─ CLAUDE.md              (project notes/instructions, not shipped code)
├─ Engine\                the reusable engine — a class library
│  ├─ Engine.csproj
│  ├─ GameWindow.cs       window + game loop + wires everything together
│  ├─ Shader.cs           compiles/links GLSL, sets uniforms
│  ├─ Texture.cs          loads a PNG into a GPU texture
│  ├─ Sprite.cs           a drawable textured quad
│  ├─ Aabb.cs             axis-aligned bounding box + intersection test
│  ├─ Player.cs           the platformer character controller
│  ├─ Shaders\            .vert / .frag GLSL source files (Content items)
│  └─ Assets\             .png textures (Content items)
└─ Game\                  the game / entry point — a console exe
   ├─ Game.csproj
   └─ Program.cs          three lines: create a GameWindow, call Run()
```

Splitting `Engine` and `Game` into two projects mirrors how almost every real engine is organized: the engine is a reusable library that knows nothing about any specific game, and the game project references it and supplies the actual content (what to draw, what the level looks like, what the player does). Right now the platformer logic still lives inside `Engine` for simplicity (see the `GameWindow.cs` section for why), but the two-project split is what makes it possible to eventually add a second, different `Game` -like project against the same `Engine` .

> **Why two projects can't be nested**
>
> An SDK-style `.csproj` (the modern .NET project format) implicitly compiles **every** `.cs` file found anywhere underneath its own folder. If `Game` 's folder were nested inside `Engine` 's folder, the `Engine` project would accidentally compile `Game` 's files too — including its own copy of the auto-generated assembly-info file — producing duplicate-attribute build errors. Keeping the two folders as siblings avoids this entirely.

The two .csproj files

Game\Game.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\Engine\Engine.csproj" />
  </ItemGroup>
</Project>
```

`OutputType=Exe` is what makes this project build to a runnable `.exe` rather than a `.dll`. `ImplicitUsings` auto-adds common `using` statements (`System`, `System.Linq`, etc.) so files don't need to repeat them. `Nullable=enable` turns on nullable reference type analysis — the compiler tracks, per variable, whether `null` is a valid value, and warns when a reference that isn't declared nullable might be null. The single `ProjectReference` is what lets `Program.cs` write `using Engine;` and construct a `GameWindow`.

Engine\Engine.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <AllowUnsafeBlocks>true</AllowUnsafeBlocks>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Silk.NET.Input" Version="2.23.0" />
    <PackageReference Include="Silk.NET.Maths" Version="2.23.0" />
    <PackageReference Include="Silk.NET.OpenGL" Version="2.23.0" />
    <PackageReference Include="Silk.NET.Windowing" Version="2.23.0" />
    <PackageReference Include="StbImageSharp" Version="2.30.15" />
  </ItemGroup>

  <ItemGroup>
    <Content Include="Shaders\**\*.*">
      <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
      <Link>Shaders\%(RecursiveDir)%(Filename)%(Extension)</Link>
    </Content>
    <Content Include="Assets\**\*.*">
      <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
      <Link>Assets\%(RecursiveDir)%(Filename)%(Extension)</Link>
    </Content>
  </ItemGroup>

</Project>
```

This is where all five NuGet packages live (see the next section for what each does). `AllowUnsafeBlocks=true` is required specifically because talking to the GPU means passing raw memory pointers around (explained in the C# features section below) — C# forbids pointer arithmetic unless a project opts in.

The final `<ItemGroup>` is what gets shader source files and PNG textures onto disk next to the compiled game, so the running program can actually find them at runtime.

> **Why the shader/asset files show up in Game's output folder automatically**
>
> These are declared as `<Content>` items (not `<None>`) with `CopyToOutputDirectory`. .NET SDK projects propagate `Content` items transitively across `ProjectReference`s — so even though these files physically live under `Engine\Shaders` and `Engine\Assets`, they get copied into `Game\bin\Debug\net10.0\Shaders\` and `...\Assets\` automatically, because `Game` references `Engine`. No duplication needed in `Game.csproj`. At runtime, `GameWindow.OnLoad` finds them via `Path.Combine(AppContext.BaseDirectory, "Shaders", ...)` — `AppContext.BaseDirectory` is the folder the running executable actually lives in.

3. The NuGet packages, and what each one buys you

Package	What it's for
`Silk.NET.Windowing`	Creates the OS window and drives the main loop. Wraps native windowing libraries (GLFW under the hood) behind a cross-platform `IWindow` interface with `Load` / `Update` / `Render` / `Resize` / `Closing` events.
`Silk.NET.OpenGL`	The actual OpenGL API binding. The `GL` class exposes essentially every OpenGL function (`GenBuffer` , `BufferData` , `DrawElements` , etc.) as ordinary C# methods, which is the whole reason this project doesn't need to write any C or use P/Invoke by hand.
`Silk.NET.Input`	Cross-platform keyboard/mouse/gamepad input, exposed via `IInputContext` . Used here for `IKeyboard.IsKeyPressed` (polling) and the `KeyDown` / `KeyUp` events (edge detection for jump buffering).
`Silk.NET.Maths`	Generic vector/matrix math types built for graphics work — e.g. `Vector2D<float>` , used throughout for positions, sizes, and velocities.
`StbImageSharp`	A C# port of the widely-used `stb_image` single-header C library. Decodes PNG (and JPG, BMP, etc.) files into raw pixel bytes that `Texture.cs` then uploads to the GPU. OpenGL itself has no idea what a PNG file is — it only understands raw pixel arrays, so something has to do this decoding step first.

One extra, non-package detail worth knowing: `System.Numerics.Matrix4x4` (used for the camera projection matrix) is part of the .NET base class library, not a NuGet package — it ships with .NET itself.

4. Core concepts primer

This section explains the graphics and engineering ideas the code relies on, so the file-by-file walkthrough later can focus on *this specific codebase's* choices rather than re-deriving OpenGL from first principles every time.

4.1 The GPU pipeline in one page

Drawing a single sprite goes through roughly this pipeline:

1. **Vertex data** — a list of numbers describing the corners of a shape (position, and here also a texture coordinate) sits in GPU memory in a **buffer**.
2. **Vertex shader** — a small GPU program runs once *per vertex*. Its job is to decide where that vertex ends up on screen (it outputs `gl_Position`).
3. **Rasterization** — fixed-function GPU hardware figures out which pixels fall inside the triangle formed by those vertices.
4. **Fragment shader** — another small GPU program runs once *per covered pixel* (a "fragment"). Its job is to decide that pixel's final color (it outputs `FragColor`).
5. **Framebuffer** — the resulting colors land in the buffer that eventually gets presented to the screen.

Both shaders are written by hand for this project (in sprite.vert and sprite.frag) — nothing about this pipeline is automatic. If you don't tell the GPU how to position a vertex or color a pixel, it has no default behavior to fall back on.

4.2 VAO, VBO, EBO — what they actually are

VBO — Vertex Buffer Object
A block of raw GPU memory holding vertex data (positions, texture coordinates, ...). Created with `GenBuffer`, filled with `BufferData`. On its own, it's just bytes — the GPU doesn't yet know how to interpret them.

Vertex attribute pointer
A description that says "starting at this byte offset, every N bytes (the *stride*), read 2 (or 3, or 4) floats — that's attribute 0 / attribute 1 / etc." Set with `VertexAttribPointer` + `EnableVertexAttribArray`. This is what turns a raw VBO into "position at location 0, texture coordinate at location 1."

VAO — Vertex Array Object

A little bit of state that *remembers* which VBO is bound and what the attribute pointer setup was. Without a VAO, you'd have to re-issue every `VertexAttribPointer` call before every single draw call. With one, you set it up once in the constructor, and drawing later is just "bind this VAO, then draw."

EBO / IBO — Element (Index) Buffer Object

A buffer of integer indices into the vertex buffer, used by `DrawElements` instead of `DrawArrays`. It lets a shape reuse vertices instead of duplicating them — see the worked example below.

> **Why `Sprite` uses 4 vertices + 6 indices instead of 6 vertices**
>
> A quad (rectangle) is two triangles. Drawn naively that's 6 vertices, but two of them (the shared diagonal corners) would be exact duplicates. Instead, `Sprite` stores just the 4 unique corners and a separate index list `{ 0, 1, 2, 2, 3, 0 }` telling the GPU "triangle one is corners 0-1-2, triangle two is corners 2-3-0." Small saving here, but the same trick matters a lot once a tilemap draws thousands of shared vertices.

4.3 Shaders, compiling, linking, uniforms

Shaders are written in **GLSL** (OpenGL Shading Language), a C-like language. Getting one running takes three steps, all visible in `Shader.cs`:

1. **Compile** the vertex shader source and the fragment shader source separately (`CreateShader` + `ShaderSource` + `CompileShader`) — each can fail independently with its own compiler error log.
2. **Link** the two compiled shaders together into one **program** (`CreateProgram` + `AttachShader` ×2 + `LinkProgram`). Linking can also fail — for example if the vertex shader's `out` variable doesn't match the fragment shader's `in` variable.
3. **Use** the program before drawing (`UseProgram`) — OpenGL is a state machine, so "use program X" just changes which program subsequent draw calls apply.

A **uniform** is a value you set from C# that stays constant across an entire draw call (as opposed to per-vertex attribute data, which varies vertex to vertex). This project uses uniforms for the sprite's position, size, the texture unit index, and the projection matrix — anything that's "one value for this whole draw," as opposed to "one value per vertex," is a uniform.

> **A subtlety that trips people up: uniforms are per-program, and "current"**
>
> `glUniform*` calls write to whichever shader program is currently bound via `UseProgram`. That's why every `SetUniform` call in this codebase happens right after `Use()` — setting a uniform before binding the program (or after binding a *different* program) would silently write to the wrong place, or nowhere useful.

4.4 Textures, texture units, filtering, mipmaps

A texture is just a 2D grid of pixel data living in GPU memory, sampled by the fragment shader via `texture(uTexture, vTexCoord)`. A few settings control how that sampling behaves:

- **Min/mag filter** — what happens when a texture is drawn smaller or larger than its native resolution. `Nearest` (used here) picks the single closest texel with no blending, which keeps flat-color/pixel-art sprites crisp instead of blurry. The alternative, `Linear`, blends neighboring texels — better for photographic textures, worse for pixel art.
- **Wrap S/T** — what happens when a texture coordinate goes outside 0..1. `ClampToEdge` (used here) just repeats the edge pixel rather than tiling or mirroring.
- **Mipmaps** — pre-shrunk copies of a texture at half, quarter, eighth resolution, etc., generated once via `GenerateMipmap`. When a texture is drawn much smaller than its native size, the GPU can sample from an appropriately-sized mip level instead of the full-resolution one, which both looks better (less aliasing) and is faster.
- **Texture unit** — the GPU has multiple texture "slots" (unit 0, unit 1, ...). `ActiveTexture(TextureUnit.Texture0)` selects a slot, `BindTexture` puts a specific texture into that slot, and the shader's `sampler2D` uniform is told *which slot number* to read from (`uTexture` is set to the integer `0`). This project only ever uses slot 0, but the mechanism exists so a shader can sample from several textures at once (e.g. a diffuse map and a normal map) in more advanced setups.

4.5 Coordinate spaces & the projection matrix

OpenGL's vertex shader must ultimately output positions in **clip space** — a fixed cube from -1 to 1 on every axis, regardless of the window's actual size or aspect ratio. Anything outside that range gets clipped away.

Hand-picking -1..1 coordinates for every sprite is unworkable once a level has real content — a 1280×720 window's platforms would need to be described as tiny fractions like `0.2`. So instead, the vertex shader works in **world space**: ordinary pixel coordinates, origin at the bottom-left, X increasing right, Y increasing up, one unit equals one pixel — the same way

you'd naturally think about a level layout. A **projection matrix** then maps that world-pixel rectangle onto the -1..1 clip cube.

This project uses an **orthographic** projection (no perspective/foreshortening — correct for 2D, where a sprite twice as far into the screen should *not* look smaller). It's built once per resize with:

```
Matrix4x4.CreateOrthographicOffCenter(0, width, 0, height, -1f, 1f)
```

which says: "map the rectangle from (0,0) to (width,height) onto clip space." Every sprite's vertex shader then does `gl_Position = uProjection * vec4(worldPosition, 0, 1)` — one matrix multiply converts a world-pixel position straight to clip space.

4.6 The game loop and deltaTime

`GameWindow` wires up three of Silk.NET's window events:

- `Load` — fires once, when the window and GL context are ready. All one-time setup (compiling shaders, loading textures, building the level) happens here.
- `Update(deltaTime)` — fires every frame, for game logic (reading input, moving the player, resolving physics).
- `Render(deltaTime)` — fires every frame, for drawing only. No game-state mutation should happen here — it should be a pure function of the current state.

> **Why every movement calculation multiplies by deltaTime**
>
> `deltaTime` is the number of seconds since the last frame. A machine running at 30 FPS calls `Update` half as often as one running at 60 FPS — so if velocity were applied as a flat "move 5 units every Update call," the 30 FPS machine would move at half speed. Multiplying by `deltaTime` (`Velocity.X * dt`) converts a rate (pixels *per second*) into an actual distance for *this particular frame*, regardless of how long that frame took — this is what makes the game run at the same real-world speed on any machine.

4.7 C# features this codebase leans on

`unsafe` / `fixed` / pointers

OpenGL is a C API — many of its functions expect a raw memory address (`void*` or a typed pointer) to read data from, because that's the only vocabulary a C API has. A C# array, by contrast, is a managed object that the garbage collector is free to move around in memory at

any time. `fixed (float* v = Vertices)` "pins" the array in place for the duration of the block, guaranteeing the GC won't relocate it while its address is being handed to unmanaged code — then `v` is a genuine pointer you can pass to `BufferData`. This is exactly what `AllowUnsafeBlocks` in the `.csproj` is unlocking.

`readonly struct`

`Aabb` is a *value type* (copied by value, not by reference, like `int` or Unity's `Vector3`) whose fields can't be modified after construction. Structs are a good fit for small, immutable math-like data — no per-instance heap allocation, and "readonly" documents that an `Aabb` is never mutated in place, only replaced.

Record-style `with` expressions

`WindowOptions.Default with { Size = ..., Title = ... }` takes an existing value and produces a modified *copy* without mutating the original — used here to start from Silk.NET's sensible defaults and only override the two settings that matter.

Expression-bodied members

Methods like `public void Use() => _gl.UseProgram(_handle);` are shorthand for a single-statement method body — purely a readability choice, identical behavior to a full `{ }` block.

Auto-properties with a private setter

`public bool IsGrounded { get; private set; }` lets any code read `IsGrounded`, but only `Player` itself can change it — an encapsulation pattern directly borrowed from how you'd expose read-only state from a Unity `MonoBehaviour`.

Lambda discard parameters

`(_, key, _) => { ... }` — when a delegate has parameters you don't care about, naming them all `_` tells the reader "this parameter is intentionally unused" more clearly than inventing throwaway names.

`MathF` instead of `Math`

`Math.Abs` / `Math.Max` /etc. operate on `double`; `MathF` is the `float`-specialized equivalent. Since this whole codebase works in `float` (matching what the GPU wants), using `MathF` avoids implicit double↔float conversions on every single call inside a function that runs every frame.

`IDisposable`

The GPU resources these classes wrap (shader programs, textures, buffers) are **not** tracked by the .NET garbage collector — from the GC's point of view, a `Shader` object is just a few bytes holding an integer handle, so it has no idea there's actual GPU memory behind it. `IDisposable.Dispose()` is the explicit "please free the real resource now" hook, called from `GameWindow.OnClosing` when the window shuts down.

5. Program.cs — the entry point

Game\Program.cs

```
using Engine;

var game = new GameWindow("My Engine - Platformer Prototype");
game.Run();
```

This is intentionally almost nothing. It's using C#'s "top-level statements" feature (no explicit `class Program { static void Main() }` boilerplate needed since C# 9) to construct a `GameWindow` with a custom title and call `Run()`, which hands control over to Silk.NET's windowing loop until the window is closed. Every actual behavior lives in the `Engine` project — this file's entire job is to be the thing `dotnet run` launches.

6. GameWindow.cs — window, input, the game loop

This is the file that owns the window and ties every other class together. Read top to bottom:

Engine\GameWindow.cs

```
using System.Numerics;
using Silk.NET.Maths;
using Silk.NET.Windowing;
using Silk.NET.OpenGL;
using Silk.NET.Input;

namespace Engine;

public class GameWindow
{
    private IWindow _window;
    private GL _gl = null!;
    private IInputContext _input = null!;
    private IKeyboard _keyboard = null!;

    private Shader _spriteShader = null!;
    private Texture _playerTexture = null!;
    private Texture _platformTexture = null!;
    private Sprite _playerSprite = null!;
    private Sprite _platformSprite = null!;

    private Player _player = null!;
    private readonly List<Aabb> _platforms = new();

    private Vector2D<int> _windowSize;
    private Matrix4x4 _projection;

    public GameWindow(string title = "My Engine", int width = 1280, int height = 720)
    {
        _windowSize = new Vector2D<int>(width, height);

        var options = WindowOptions.Default with
        {
            Size  = _windowSize,
            Title = title
        };

        _window = Window.Create(options);

        _window.Load     += OnLoad;    // runs once, when the window is ready
        _window.Update    += OnUpdate; // runs every frame: game logic
        _window.Render    += OnRender; // runs every frame: drawing
```

```
        _window.Resize    += OnResize;
        _window.Closing   += OnClosing;
    }

    public void Run() => _window.Run();

    private void OnLoad()
    {
        _gl = GL.GetApi(_window);
        _gl.ClearColor(0.53f, 0.72f, 0.90f, 1.0f); // sky blue — nicer backdrop for a platformer
        _gl.Enable(EnableCap.Blend);
        _gl.BlendFunc(GLEnum.SrcAlpha, GLEnum.OneMinusSrcAlpha);

        _input = _window.CreateInput();
        _keyboard = _input.Keyboards[0];

        string shaderDir = Path.Combine(AppContext.BaseDirectory, "Shaders");
        string assetsDir = Path.Combine(AppContext.BaseDirectory, "Assets");

        _spriteShader = new Shader(_gl,
            Path.Combine(shaderDir, "sprite.vert"),
            Path.Combine(shaderDir, "sprite.frag"));

        _playerTexture = new Texture(_gl, Path.Combine(assetsDir, "player.png"));
        _platformTexture = new Texture(_gl, Path.Combine(assetsDir, "platform.png"));
        _playerSprite = new Sprite(_gl, _spriteShader, _playerTexture);
        _platformSprite = new Sprite(_gl, _spriteShader, _platformTexture);

        _player = new Player { Position = new Vector2D<float>(100f, 64f) };

        BuildLevel();
        UpdateProjection();

        _keyboard.KeyDown += (_, key, _) => { if (IsJumpKey(key)) _player.BufferJump(); };
        _keyboard.KeyUp   += (_, key, _) => { if (IsJumpKey(key)) _player.CutJumpIfRising(); };
    }

    private void BuildLevel()
    {
        // Hand-placed rectangles for now — this is the AABB collision half
        // of Episode 5, done ahead of Episode 4's proper tile-grid loader.
        _platforms.Add(new Aabb(new Vector2D<float>(0f, 0f), new Vector2D<float>(1280f, 64f)));
// ground
        _platforms.Add(new Aabb(new Vector2D<float>(250f, 160f), new Vector2D<float>(200f,
32f)));
        _platforms.Add(new Aabb(new Vector2D<float>(550f, 280f), new Vector2D<float>(200f,
32f)));
        _platforms.Add(new Aabb(new Vector2D<float>(850f, 200f), new Vector2D<float>(220f,
32f)));
        _platforms.Add(new Aabb(new Vector2D<float>(1120f, 340f), new Vector2D<float>(160f,
32f)));
    }
```

```
    private static bool IsJumpKey(Key key) => key is Key.Space or Key.W or Key.Up;

    private void OnUpdate(double deltaTime)
    {
        float moveInput = 0f;
        if (_keyboard.IsKeyPressed(Key.A) || _keyboard.IsKeyPressed(Key.Left)) moveInput -= 1f;
        if (_keyboard.IsKeyPressed(Key.D) || _keyboard.IsKeyPressed(Key.Right)) moveInput += 1f;

        _player.Update((float)deltaTime, moveInput, _platforms);
    }

    private void OnRender(double deltaTime)
    {
        _gl.Clear(ClearBufferMask.ColorBufferBit);

        foreach (var platform in _platforms)
            _platformSprite.Draw(platform.Center, platform.Size, _projection);

        var playerCenter = new Vector2D<float>(
            _player.Position.X + _player.Size.X / 2f,
            _player.Position.Y + _player.Size.Y / 2f);
        _playerSprite.Draw(playerCenter, _player.Size, _projection);
    }

    private void OnResize(Vector2D<int> size)
    {
        _windowSize = size;
        _gl.Viewport(size);
        UpdateProjection();
    }

    private void UpdateProjection()
    {
        // World space: origin bottom-left, X right, Y up, 1 unit = 1 pixel —
        // ordinary screen-coordinate thinking, unlike raw GL clip space.
        _projection = Matrix4x4.CreateOrthographicOffCenter(0, _windowSize.X, 0, _windowSize.Y,
-1f, 1f);
    }

    private void OnClosing()
    {
        _playerSprite.Dispose();
        _platformSprite.Dispose();
        _playerTexture.Dispose();
        _platformTexture.Dispose();
        _spriteShader.Dispose();
        _input.Dispose();
    }
}
```

Walkthrough

Fields (lines 11–26). The `= null!` pattern on several fields is a deliberate promise to the nullable-reference-type checker: "this will be non-null by the time anything uses it, even though I can't assign it in the constructor" (it's actually assigned in `OnLoad`, which the compiler can't statically prove runs first — Silk.NET guarantees it at runtime). `_platforms` is a plain `List<Aabb>` — the level's static collision geometry.

Constructor (lines 28–45). Builds a `WindowOptions` value (using the `with` expression explained earlier), creates the actual OS window via `Window.Create`, and subscribes to five of its lifecycle events. Note that creating the window here does *not* yet create an OpenGL context or let you call any GL functions — that only becomes valid once `Load` fires.

`OnLoad` **(lines 49–78).** Everything one-time:

- `GL.GetApi(_window)` — now that the window exists, grab the actual OpenGL function table.
- `ClearColor` — the RGBA color used whenever the screen is wiped each frame (sky blue, as 0..1 floats, not 0..255 bytes).
- `Enable(EnableCap.Blend)` + `BlendFunc(...)` — turns on alpha blending using the standard "src-over" formula (`finalColor = srcColor × srcAlpha + dstColor × (1 - srcAlpha)`), so any sprite with a transparent PNG background composites correctly instead of drawing a solid box.
- Input context + keyboard reference are grabbed once and kept.
- Shader, both textures, and both sprites are constructed — see their own sections.
- The `Player` is created with a starting position standing on the ground (`y = 64`, matching the ground platform's height).
- `BuildLevel()` and `UpdateProjection()` populate the level and the camera matrix.
- Finally, `KeyDown` / `KeyUp` are wired to the jump-buffer and jump-cut hooks on `Player` — explained fully in the Player.cs section.

`BuildLevel` **(lines 80–89).** Five hardcoded `Aabb` rectangles: a wide ground strip and four floating platforms at increasing heights, laid out so a player standing on one can reach the next by jumping. This stands in for a real tile-grid loader (Episode 4), which isn't built yet.

`OnUpdate` **(lines 93–100).** Polls the keyboard every frame for continuous left/right movement (`IsKeyPressed` checks "is this key held down right now", unlike the edge-triggered `KeyDown` event) and hands the resulting -1/0/+1 `moveInput` value, plus `deltaTime` and the level geometry, to `Player.Update`. All the actual physics/collision logic lives inside `Player` itself — `GameWindow` only supplies input and geometry.

`OnRender` **(lines 102–113).** Clears the screen, then draws every platform followed by the player. Note `Sprite.Draw` takes a *center* position, but `Aabb` and `Player.Position` both

store the *bottom-left* corner — so the player's draw position is computed as `Position + Size / 2` each frame, while `Aabb.Center` does the equivalent conversion for platforms.

`OnResize` / `UpdateProjection` **(lines 115–127).** If the window is resized, the GL viewport (the region of the window OpenGL actually renders into) and the projection matrix both need to change, or sprites would end up stretched or misaligned. Both are recomputed from the new size.

`OnClosing` **(lines 129–137).** Explicit cleanup of every GPU-backed resource created in `OnLoad`, in roughly reverse order of creation — this is the "why `IDisposable` matters" point made concrete: none of this happens automatically.

7. Shader.cs — compiling and talking to a GL program

Engine\Shader.cs

```
using Silk.NET.OpenGL;
using Silk.NET.Maths;
using System.Numerics;

namespace Engine;

// Wraps a compiled + linked GL shader program. Extracted out of GameWindow
// in Episode 3 so both the triangle demo and Sprite can reuse it instead of
// duplicating compile/link boilerplate.
public class Shader : IDisposable
{
    private readonly GL _gl;
    private readonly uint _handle;

    public Shader(GL gl, string vertexPath, string fragmentPath)
    {
        _gl = gl;

        uint vertex = Compile(ShaderType.VertexShader, File.ReadAllText(vertexPath));
        uint fragment = Compile(ShaderType.FragmentShader, File.ReadAllText(fragmentPath));

        _handle = _gl.CreateProgram();
        _gl.AttachShader(_handle, vertex);
        _gl.AttachShader(_handle, fragment);
        _gl.LinkProgram(_handle);

        _gl.GetProgram(_handle, ProgramPropertyARB.LinkStatus, out int linkStatus);
        if (linkStatus == 0)
            throw new Exception($"Shader program failed to link:
{_gl.GetProgramInfoLog(_handle)}");

        // The program keeps its own copy once linked; the individual shader
        // objects aren't needed after that.
        _gl.DetachShader(_handle, vertex);
        _gl.DetachShader(_handle, fragment);
        _gl.DeleteShader(vertex);
        _gl.DeleteShader(fragment);
    }

    private uint Compile(ShaderType type, string source)
    {
        uint id = _gl.CreateShader(type);
        _gl.ShaderSource(id, source);
        _gl.CompileShader(id);
```

```
        _gl.GetShader(id, ShaderParameterName.CompileStatus, out int compileStatus);
        if (compileStatus == 0)
            throw new Exception($"{type} failed to compile: {_gl.GetShaderInfoLog(id)}");

        return id;
    }

    public void Use() => _gl.UseProgram(_handle);

    public void SetUniform(string name, int value)
    {
        _gl.Uniform1(_gl.GetUniformLocation(_handle, name), value);
    }

    public void SetUniform(string name, float value)
    {
        _gl.Uniform1(_gl.GetUniformLocation(_handle, name), value);
    }

    public void SetUniform(string name, Vector2D<float> value)
    {
        _gl.Uniform2(_gl.GetUniformLocation(_handle, name), value.X, value.Y);
    }

    // System.Numerics.Matrix4x4 (not Silk.NET.Maths) purely because it's a
    // plain sequential struct of 16 floats, so `&value` is a valid float*
    // straight into GL — no separate float[16] copy needed.
    public unsafe void SetUniform(string name, Matrix4x4 value)
    {
        int location = _gl.GetUniformLocation(_handle, name);
        _gl.UniformMatrix4(location, 1, false, (float*)&value);
    }

    public void Dispose() => _gl.DeleteProgram(_handle);
}
```

Walkthrough

The constructor is a direct implementation of the three-step compile/link/use process from the concepts primer. Two details worth calling out:

- **Error checking happens twice, separately** — once per shader (`CompileStatus`) and once for the linked program (`LinkStatus`). A shader can compile fine on its own but still fail to link (e.g. mismatched `in` / `out` variable names between vertex and fragment stages), so both checks matter, and both throw an exception carrying the GPU driver's own error log text (`GetShaderInfoLog` / `GetProgramInfoLog`) — invaluable for debugging a shader typo.

- **Detach + delete after linking** — once a program is linked, it has its own internal copy of the compiled shader code. The intermediate shader objects are pure overhead after that point, so they're cleaned up immediately rather than held onto for the lifetime of the `Shader` instance.

`SetUniform` is **overloaded** four times for the four uniform types this project actually needs (`int`, `float`, a 2D vector, and a 4×4 matrix) — C# picks the right overload automatically based on the argument type at the call site, so callers just write `SetUniform("uSize", someVector)` without caring which underlying `glUniform*` function that maps to.

The matrix overload is the one place this class needs `unsafe`: `UniformMatrix4` wants a `float*` pointing at 16 contiguous floats. `System.Numerics.Matrix4x4` happens to be laid out in memory as exactly that (16 sequential `float` fields named `M11` through `M44`), so `&value` — "the address of this local variable" — is already a valid pointer to feed GL directly, with no need to manually copy the matrix into a `float[16]` array first.

8. Texture.cs — loading a PNG onto the GPU

Engine\Texture.cs

```
using Silk.NET.OpenGL;
using StbImageSharp;

namespace Engine;

// Loads a PNG/JPG off disk into a GL texture. This is what turns a plain
// textured quad (Sprite) into something that actually looks like art.
public class Texture : IDisposable
{
    private readonly GL _gl;
    private readonly uint _handle;

    public unsafe Texture(GL gl, string path)
    {
        _gl = gl;
        _handle = _gl.GenTexture();
        Bind();

        // OpenGL expects row 0 at the bottom of the image; PNGs store row 0
        // at the top. Flipping on load means UV (0,0) is the bottom-left,
        // matching the convention every OpenGL tutorial assumes.
        StbImage.stbi_set_flip_vertically_on_load(1);

        using var stream = File.OpenRead(path);
        var image = ImageResult.FromStream(stream, ColorComponents.RedGreenBlueAlpha);

        fixed (byte* pixels = image.Data)
        {
            _gl.TexImage2D(
                TextureTarget.Texture2D,
                0,
                InternalFormat.Rgba,
                (uint)image.Width,
                (uint)image.Height,
                0,
                PixelFormat.Rgba,
                PixelType.UnsignedByte,
                pixels);
        }

        // Nearest-neighbor filtering keeps pixel art crisp instead of blurry.
        _gl.TexParameter(TextureTarget.Texture2D, TextureParameterName.TextureMinFilter,
(int)GLEnum.Nearest);
        _gl.TexParameter(TextureTarget.Texture2D, TextureParameterName.TextureMagFilter,
(int)GLEnum.Nearest);
        _gl.TexParameter(TextureTarget.Texture2D, TextureParameterName.TextureWrapS,
(int)GLEnum.ClampToEdge);
```

```
        _gl.TexParameter(TextureTarget.Texture2D, TextureParameterName.TextureWrapT,
(int)GLEnum.ClampToEdge);

        _gl.GenerateMipmap(TextureTarget.Texture2D);
    }

    public void Bind(TextureUnit unit = TextureUnit.Texture0)
    {
        _gl.ActiveTexture(unit);
        _gl.BindTexture(TextureTarget.Texture2D, _handle);
    }

    public void Dispose() => _gl.DeleteTexture(_handle);
}
```

Walkthrough

`GenTexture` allocates a texture handle; `Bind()` makes it the "current" 2D texture so subsequent calls (`TexImage2D` , `TexParameter`) apply to it — the same bind-then-configure pattern OpenGL uses everywhere.

`stbi_set_flip_vertically_on_load(1)` exists because of a long-standing mismatch between image file formats (which store the top row of pixels first) and OpenGL's texture coordinate convention (which treats `(0,0)` as the **bottom**-left of a texture). Flipping at load time means texture coordinates in the shader behave the way every tutorial and every other engine assumes, rather than needing to flip UVs everywhere a texture gets sampled.

`ImageResult.FromStream(..., ColorComponents.RedGreenBlueAlpha)` forces the decoded output to always be 4 bytes per pixel (R, G, B, A) regardless of the source PNG's actual format (even if it had no alpha channel) — this keeps the upload code below simple and uniform, since it can always assume RGBA.

`fixed (byte* pixels = image.Data)` pins the decoded pixel array (the same technique as `Sprite` 's vertex data) so `TexImage2D` — the actual "upload these pixels to the GPU" call — can be handed a raw pointer. `TexImage2D` 's parameters describe the texture's target, mip level (`0` = the full-resolution base image), internal storage format, dimensions, and the format/type of the pixel data being uploaded.

The four `TexParameter` calls and the primer's explanations of filtering/wrapping apply directly here — nearest-neighbor filtering plus edge clamping, chosen because both current textures (a solid-color player square and a solid-color platform) are simple flat colors where crisp edges matter more than smooth interpolation.

9. Sprite.cs — a drawable textured quad

Engine\Sprite.cs

```
using Silk.NET.OpenGL;
using Silk.NET.Maths;
using System.Numerics;

namespace Engine;

// A textured quad: two triangles sharing an edge, drawn via an index buffer
// so the shared vertices aren't duplicated. This is the first real "sprite"
// in the engine — Episode 4's tilemaps will just be many of these.
public class Sprite : IDisposable
{
    // Unit quad centered on the origin; Draw() scales/moves it via uniforms
    // rather than baking a position into the vertex data, so one Sprite
    // instance's GPU buffers can be reused to draw at any position/size.
    private static readonly float[] Vertices =
    {
        // position       // texcoord
        -0.5f,  0.5f,     0f, 1f, // top-left
         0.5f,  0.5f,     1f, 1f, // top-right
         0.5f, -0.5f,     1f, 0f, // bottom-right
        -0.5f, -0.5f,     0f, 0f, // bottom-left
    };

    private static readonly uint[] Indices = { 0, 1, 2, 2, 3, 0 };

    private readonly GL _gl;
    private readonly Shader _shader;
    private readonly Texture _texture;
    private readonly uint _vao;
    private readonly uint _vbo;
    private readonly uint _ebo;

    public unsafe Sprite(GL gl, Shader shader, Texture texture)
    {
        _gl = gl;
        _shader = shader;
        _texture = texture;

        _vao = _gl.GenVertexArray();
        _gl.BindVertexArray(_vao);

        _vbo = _gl.GenBuffer();
        _gl.BindBuffer(BufferTargetARB.ArrayBuffer, _vbo);
        fixed (float* v = Vertices)
            _gl.BufferData(BufferTargetARB.ArrayBuffer, (nuint)(Vertices.Length * sizeof(float)),
v, BufferUsageARB.StaticDraw);
```

```
        _ebo = _gl.GenBuffer();
        _gl.BindBuffer(BufferTargetARB.ElementArrayBuffer, _ebo);
        fixed (uint* i = Indices)
            _gl.BufferData(BufferTargetARB.ElementArrayBuffer, (nuint)(Indices.Length *
sizeof(uint)), i, BufferUsageARB.StaticDraw);

        const uint stride = 4 * sizeof(float);
        _gl.VertexAttribPointer(0, 2, VertexAttribPointerType.Float, false, stride, (void*)0);
        _gl.EnableVertexAttribArray(0);
        _gl.VertexAttribPointer(1, 2, VertexAttribPointerType.Float, false, stride, (void*)(2 *
sizeof(float)));
        _gl.EnableVertexAttribArray(1);

        _gl.BindVertexArray(0);
    }

    public unsafe void Draw(Vector2D<float> position, Vector2D<float> size, Matrix4x4 projection)
    {
        _shader.Use();
        _shader.SetUniform("uPosition", position);
        _shader.SetUniform("uSize", size);
        _shader.SetUniform("uProjection", projection);

        _texture.Bind(TextureUnit.Texture0);
        _shader.SetUniform("uTexture", 0);

        _gl.BindVertexArray(_vao);
        _gl.DrawElements(PrimitiveType.Triangles, (uint)Indices.Length,
DrawElementsType.UnsignedInt, null);
    }

    public void Dispose()
    {
        _gl.DeleteVertexArray(_vao);
        _gl.DeleteBuffer(_vbo);
        _gl.DeleteBuffer(_ebo);
    }
}
```

Walkthrough

`Vertices` describes a quad from -0.5 to 0.5 on both axes — a "unit square" centered on the origin — interleaved as `x, y, u, v` per vertex (position immediately followed by that same vertex's texture coordinate). Notice the texture coordinates: the top-left vertex maps to UV `(0, 1)` and the bottom-left to `(0, 0)` — consistent with the flipped-on-load convention from `Texture.cs`, where `v = 0` really is the bottom of the image.

The constructor performs the full VAO/VBO/EBO setup dance from the primer, once, when a `Sprite` is created:

- Generate and bind a VAO first — everything configured afterward gets recorded into it.
- Generate a VBO, bind it as `ArrayBuffer`, upload `Vertices`.
- Generate an EBO, bind it as `ElementArrayBuffer`, upload `Indices`.
- Describe the interleaved layout with two `VertexAttribPointer` calls: attribute 0 (position) is 2 floats starting at byte offset 0; attribute 1 (texcoord) is 2 floats starting at byte offset 8 (`2 * sizeof(float)`) — both with a `stride` of 16 bytes (4 floats total per vertex), which tells the GPU how far to jump to reach the *next* vertex's data.

`Draw` is what runs every frame for every sprite: bind the shader, push this particular sprite's position/size/projection as uniforms, bind its texture into slot 0, bind its VAO (restoring all the attribute-pointer state configured in the constructor), and issue the actual draw call. `DrawElements` (as opposed to `DrawArrays`) is what makes the GPU walk the index buffer instead of the vertex buffer directly — 6 indices, producing 2 triangles, referencing only 4 unique vertices.

> **Why position/size are uniforms instead of baked into the vertex data**
>
> If the quad's actual on-screen position were baked directly into `Vertices`, a `Sprite` instance could only ever be drawn in one place — moving it would mean re-uploading new vertex data to the GPU every frame. By keeping the geometry as a generic unit square and applying position/size in the vertex shader via uniforms instead, the exact same GPU buffers get reused to draw the player at one position this frame and a platform at a completely different position/size the next — no re-upload needed, just a cheap uniform write.

10. sprite.vert / sprite.frag — the actual GLSL

Engine\Shaders\sprite.vert

```
#version 330 core

// A unit quad (-0.5..0.5) is stretched by uSize and moved by uPosition, both
// in world-space pixels (origin bottom-left, Y up). uProjection then maps
// that world-pixel space onto the -1..1 clip space GL actually wants — this
// is what lets platform/player positions be specified in ordinary pixel
// coordinates instead of hand-picked NDC fractions.
layout (location = 0) in vec2 aPosition;
layout (location = 1) in vec2 aTexCoord;

uniform vec2 uPosition;
uniform vec2 uSize;
uniform mat4 uProjection;

out vec2 vTexCoord;

void main()
{
    vec2 worldPosition = aPosition * uSize + uPosition;
    gl_Position = uProjection * vec4(worldPosition, 0.0, 1.0);
    vTexCoord = aTexCoord;
}
```

Engine\Shaders\sprite.frag

```
#version 330 core

in vec2 vTexCoord;
out vec4 FragColor;

uniform sampler2D uTexture;

void main()
{
    FragColor = texture(uTexture, vTexCoord);
}
```

Walkthrough

`#version 330 core` targets OpenGL 3.3's "core" (non-deprecated) profile — the baseline this whole project builds against.

`layout (location = 0) in vec2 aPosition;` and `layout (location = 1) in vec2 aTexCoord;` are the **inputs** to the vertex shader — one pair of values per vertex, sourced from

whatever `VertexAttribPointer` location 0 and location 1 were configured to read from (that's `Sprite.cs`'s job, matching attribute locations to these declarations). The three `uniform` declarations are the per-draw-call constants set from C# via `Shader.SetUniform`.

`main()` computes `worldPosition` by scaling the unit quad by `uSize` and shifting it by `uPosition` — this is why a `Sprite` instance can represent both a 48×48 player and a 200×32 platform: the same `-0.5..0.5` geometry, stretched differently per draw call. It then multiplies by `uProjection` to land in clip space, and passes the unmodified texture coordinate through to `vTexCoord` — the `out` here matching the fragment shader's `in` of the same name is what "links" the two stages together (mismatched names between stages is a linker error, as mentioned in the `Shader.cs` walkthrough).

The fragment shader is about as simple as one can be: for every pixel this quad covers, sample the bound texture at the (automatically interpolated) texture coordinate and use that as the final pixel color, unmodified. "Interpolated" is doing real work here — even though `vTexCoord` is only explicitly set at the 4 corners in the vertex shader, the GPU automatically computes intermediate values for every pixel in between during rasterization, which is exactly what lets a single texture stretch smoothly across an entire quad.

11. Aabb.cs — axis-aligned bounding boxes

Engine\Aabb.cs

```
using Silk.NET.Maths;

namespace Engine;

// Axis-aligned bounding box: Min is the bottom-left corner, in the same
// world-pixel, Y-up space the camera projection uses. Used for both level
// geometry (static platforms) and the player's own collision box.
public readonly struct Aabb
{
    public readonly Vector2D<float> Min;
    public readonly Vector2D<float> Size;

    public Aabb(Vector2D<float> min, Vector2D<float> size)
    {
        Min = min;
        Size = size;
    }

    public Vector2D<float> Max => new(Min.X + Size.X, Min.Y + Size.Y);
    public Vector2D<float> Center => new(Min.X + Size.X / 2f, Min.Y + Size.Y / 2f);

    // Strict inequalities: two boxes exactly touching edge-to-edge (as
    // happens right after collision resolution pushes the player flush
    // against a platform) do NOT count as intersecting.
    public bool Intersects(Aabb other) =>
        Min.X < other.Max.X && Max.X > other.Min.X &&
        Min.Y < other.Max.Y && Max.Y > other.Min.Y;
}
```

Walkthrough

"Axis-aligned" means the rectangle's edges are always parallel to the X and Y axes — no rotation. This is the simplest possible 2D collision shape, and is exactly why it's the right starting point for a platformer: platforms and characters in 2D platformers are almost always rectangles anyway.

An `Aabb` stores only `Min` (bottom-left corner) and `Size` — everything else (`Max`, `Center`) is derived arithmetic exposed as computed properties (`=>` getters with no backing field), so there's only ever one source of truth to keep consistent.

`Intersects` implements the standard 2D AABB overlap test: two rectangles overlap if and only if they overlap on *both* the X axis and the Y axis independently. Each axis check (`Min.X <`

`other.Max.X && Max.X > other.Min.X`) reads as "this box's left edge is left of the other's right edge, AND this box's right edge is right of the other's left edge" — if either axis fails that test, the boxes can't possibly overlap, no matter what the other axis says.

> **Why strict `<` / `>` instead of `<=` / `>=`**
>
> After `Player` resolves a collision, it pushes the player's box to sit exactly flush against the platform it just hit — edges touching, zero gap, zero overlap. If `Intersects` used `<=` / `>=` , that flush-touching state would itself count as "intersecting," and the very next frame's collision check would think the player is still embedded in the platform it's actually just standing on. Strict inequality means "touching is fine, only genuine overlap counts" — which is what makes standing on a platform stable frame after frame instead of jittering.

12. Player.cs — the platformer controller, in depth

This is the file that actually implements everything asked for: smooth acceleration/deceleration, gravity, jump buffering, coyote time, and collision. It's also the one place in the project doing real per-frame gameplay math rather than graphics plumbing, so it gets the most detailed walkthrough.

Engine\Player.cs

```
using Silk.NET.Maths;

namespace Engine;

// A platformer character controller. Two things make it feel "smooth"
// instead of instant/twitchy:
//  1. Horizontal velocity eases toward a target speed (Acceleration) and
//     eases back to zero (Deceleration) instead of snapping, like Update()
//     lerping a Rigidbody2D.velocity in Unity.
//  2. Two forgiveness windows around jumping: coyote time (still jumpable
//     for a few frames after walking off a ledge) and jump buffering (a
//     jump press a few frames before landing isn't dropped, it fires the
//     instant you touch ground). Both are just countdown timers.
public class Player
{
    public Vector2D<float> Position; // bottom-left corner, world pixels
    public Vector2D<float> Velocity;
    public Vector2D<float> Size = new(48f, 48f);
    public bool IsGrounded { get; private set; }

    // Tuning knobs, in pixels/second and pixels/second^2. Public so they're
    // easy to tweak live while chasing "game feel" without touching logic.
    public float MoveSpeed = 420f;
    public float Acceleration = 4000f;
    public float Deceleration = 5000f;
    public float AirControlMultiplier = 0.7f; // less responsive while airborne
    public float Gravity = 2200f;
    public float MaxFallSpeed = 1400f;
    public float JumpVelocity = 900f;
    public float JumpCutMultiplier = 0.45f; // releasing jump early cuts the rise short
    public float CoyoteTime = 0.1f;
    public float JumpBufferTime = 0.12f;

    private float _coyoteTimer;
    private float _jumpBufferTimer;

    // Call when the jump key is pressed. Doesn't jump immediately — just
    // starts the buffer countdown; Update() decides whether it's allowed.
```

```
    public void BufferJump() => _jumpBufferTimer = JumpBufferTime;

    // Call when the jump key is released. A short hop if we're still rising.
    public void CutJumpIfRising()
    {
        if (Velocity.Y > 0f)
            Velocity.Y *= JumpCutMultiplier;
    }

    public void Update(float dt, float moveInput, IReadOnlyList<Aabb> solids)
    {
        float targetSpeed = moveInput * MoveSpeed;
        float rate = MathF.Abs(moveInput) > 0.01f ? Acceleration : Deceleration;
        if (!IsGrounded) rate *= AirControlMultiplier;
        Velocity.X = MoveTowards(Velocity.X, targetSpeed, rate * dt);

        // Coyote timer resets every frame you're grounded, then drains once
        // you leave the ground. Jump buffer just drains every frame; it's
        // (re)armed from outside via BufferJump().
        _coyoteTimer = IsGrounded ? CoyoteTime : MathF.Max(0f, _coyoteTimer - dt);
        _jumpBufferTimer = MathF.Max(0f, _jumpBufferTimer - dt);

        if (_jumpBufferTimer > 0f && _coyoteTimer > 0f)
        {
            Velocity.Y = JumpVelocity;
            _jumpBufferTimer = 0f;
            _coyoteTimer = 0f;
            IsGrounded = false;
        }

        Velocity.Y = MathF.Max(Velocity.Y - Gravity * dt, -MaxFallSpeed);

        // Resolve X and Y separately rather than moving diagonally and
        // testing once — otherwise sliding into a wall while falling could
        // shove the player through the floor along with it.
        MoveAndCollide(new Vector2D<float>(Velocity.X * dt, 0f), solids);
        IsGrounded = false;
        MoveAndCollide(new Vector2D<float>(0f, Velocity.Y * dt), solids);
    }

    private void MoveAndCollide(Vector2D<float> delta, IReadOnlyList<Aabb> solids)
    {
        Position = new Vector2D<float>(Position.X + delta.X, Position.Y + delta.Y);
        var box = new Aabb(Position, Size);

        foreach (var solid in solids)
        {
            if (!box.Intersects(solid)) continue;

            float overlapX = MathF.Min(box.Max.X, solid.Max.X) - MathF.Max(box.Min.X,
solid.Min.X);
            float overlapY = MathF.Min(box.Max.Y, solid.Max.Y) - MathF.Max(box.Min.Y,
solid.Min.Y);
```

```
            if (delta.X != 0f)
            {
                float pushX = delta.X > 0f ? -overlapX : overlapX;
                Position = new Vector2D<float>(Position.X + pushX, Position.Y);
                Velocity = new Vector2D<float>(0f, Velocity.Y);
            }
            else if (delta.Y != 0f)
            {
                if (delta.Y < 0f) IsGrounded = true;
                float pushY = delta.Y > 0f ? -overlapY : overlapY;
                Position = new Vector2D<float>(Position.X, Position.Y + pushY);
                Velocity = new Vector2D<float>(Velocity.X, 0f);
            }

            box = new Aabb(Position, Size);
        }
    }

    private static float MoveTowards(float current, float target, float maxDelta)
    {
        if (MathF.Abs(target - current) <= maxDelta) return target;
        return current + MathF.Sign(target - current) * maxDelta;
    }
}
```

Tuning fields

Every "feel" constant is a public field with a sensible default, deliberately exposed so they can be tweaked without touching any logic — the same philosophy as exposing tunable values in a Unity Inspector. `MoveSpeed` is the top horizontal speed in pixels/second; `Acceleration` / `Deceleration` are in pixels/second², i.e. how quickly velocity itself changes; `Gravity` and `MaxFallSpeed` control the fall; `JumpVelocity` is the instantaneous upward speed applied at the moment of a jump.

`MoveTowards` — the acceleration/deceleration engine

This is a direct port of the technique behind Unity's `Mathf.MoveTowards`: move `current` toward `target` by at most `maxDelta`, and snap exactly to `target` if the remaining distance is smaller than that step (so it never overshoots and oscillates around the target). In `Update`, the "target" is `moveInput * MoveSpeed` — full speed left, full speed right, or (with no input) zero — and the "step size" (`rate * dt`) uses `Acceleration` while there's input and `Deceleration` while there isn't. This single function is what turns movement from an instant on/off snap into a smooth ramp up and ramp down.

`AirControlMultiplier` (0.7 by default) shrinks that same rate while airborne — still using the identical `MoveTowards` curve, just slower to respond, which is the classic "floatier in the air" platformer feel.

Coyote time and jump buffering

Both are implemented as nothing more than two countdown timers (`_coyoteTimer`, `_jumpBufferTimer`), decremented every frame by `dt` and clamped at zero via `MathF.Max(0f, ...)`. What makes them do their job is *when* each one gets reset:

- `_coyoteTimer` is reset to full (`CoyoteTime`) every single frame the player is grounded, and only starts counting down the moment `IsGrounded` becomes false. So "is `_coyoteTimer > 0`" really means "was grounded within the last `CoyoteTime` seconds."
- `_jumpBufferTimer` is reset only by an external call to `BufferJump()` — wired in `GameWindow` to the jump key's `KeyDown` event. So "is `_jumpBufferTimer > 0`" means "jump was pressed within the last `JumpBufferTime` seconds."

The actual jump only fires when **both** conditions hold at once (`_jumpBufferTimer > 0f && _coyoteTimer > 0f`) — a recently-pressed jump AND a recently-grounded player. See the worked examples section for a frame-by-frame timeline of exactly why this combination is what gives both forgiveness effects at once.

Jump cut (variable jump height)

`CutJumpIfRising`, wired to the jump key's `KeyUp` event, multiplies the current upward velocity by `JumpCutMultiplier` (0.45) — but only if the player is still moving upward (`Velocity.Y > 0`). Releasing jump early cuts the rise short for a small hop; holding it through the peak has no effect (by the time you'd release it, `Velocity.Y` is already at or past zero from gravity), giving the full-height jump. This is the same mechanic as the classic "tap vs. hold" jump height found in most platformers.

Gravity

`Velocity.Y = MathF.Max(Velocity.Y - Gravity * dt, -MaxFallSpeed)` integrates gravity every frame — subtract `Gravity * dt` from the current vertical velocity — and clamps the result so falling never exceeds `MaxFallSpeed`. Without that clamp, a long fall would keep accelerating indefinitely, which both looks wrong and can cause a fast-enough object to tunnel through thin platforms in a single frame's movement.

`MoveAndCollide` — the collision resolution algorithm

This is called twice per `Update`: once with a purely horizontal `delta` (`Velocity.X * dt`, no Y component) and once with a purely vertical one (`Velocity.Y * dt`, no X component) — never both at once. The function:

1. Moves `Position` by the given `delta` unconditionally.
2. Builds an `Aabb` at the new position and checks it against every solid in the level.
3. For any solid it now overlaps, computes how much it's overlapping on each axis (`overlapX`, `overlapY` — the size of the intersection rectangle).
4. Since this function is only ever moving one axis at a time, it knows which correction to apply: if this was the X-pass (`delta.X != 0`), push the player back out along X by `overlapX`, in whichever direction undoes the overlap, and zero `Velocity.X` (the wall stopped the player's horizontal motion this frame). If it was the Y-pass, do the same along Y — and additionally, if the player was moving *downward* when it happened (`delta.Y < 0`), that means it just landed on something, so `IsGrounded` is set true.

Note that `IsGrounded` is reset to `false` right before the Y-axis pass (back in `Update`, between the two `MoveAndCollide` calls) and only becomes `true` again if that pass actually detects a downward collision — it is **recomputed from scratch every single frame**, never just assumed to still be true from the previous frame. That's what makes walking off a ledge correctly and immediately set `IsGrounded = false` (which is, in turn, exactly what starts the coyote timer counting down).

> **Why resolve X and Y in two separate passes instead of one diagonal move**
>
> Imagine the player is falling fast while also holding a direction key toward a wall. Moving diagonally in one step and testing collision once could, depending on exact numbers, land the corner of the player's box past both the wall *and* the floor simultaneously, and a single combined correction has no unambiguous way to decide "push back left" vs. "push back up" — different implementations of that would give different (often wrong) results, like corner-clipping through geometry. Resolving one axis fully (and correctly) before even starting the other guarantees each axis's correction only ever has to reason about a single direction's worth of overlap, which is simpler and far more robust for axis-aligned rectangles specifically.

13. Game-feel mechanics — worked examples

Assume 60 FPS, so each frame is about 0.0167 seconds, and the defaults from `Player.cs`: `CoyoteTime = 0.1s` (~6 frames), `JumpBufferTime = 0.12s` (~7 frames).

Coyote time timeline

Frame	Event	_coyoteTimer	Can still jump?
0	Standing on a platform, walks forward off the edge	0.1 (reset every grounded frame)	Yes (grounded)
1	`IsGrounded` is now false	0.083	Yes — within window
4	Still falling, no ground under feet	0.033	Yes — within window
7	Timer has drained to 0	0.0	No — window closed

A jump press landing anywhere in frames 0–6 above still succeeds, even though the player has visibly already walked off the platform — this is what makes an almost-too-late jump input feel fair instead of punishing.

Jump buffering timeline

Frame	Event	_jumpBufferTimer	_coyoteTimer	Jump fires?
0	Still airborne, jump key pressed → `BufferJump()`	0.12	0.0 (airborne)	No — not grounded, no coyote window
1–3	Still falling toward the platform	draining	0.0	No
4	Touches the platform — `IsGrounded` becomes true	~0.053 (still > 0)	resets to 0.1	**Yes** — both timers > 0 this frame

Without buffering, that frame-0 press would simply be lost (the player wasn't grounded yet, so a naive "jump only if grounded and key pressed" check would never see it land at the right moment) — the player would need to press jump *again* after landing, which reads as an unresponsive, missed input.

Axis-separated collision, worked example

A player falling fast (`Velocity = (250, -900)` px/s) approaches the top-left corner of a platform. In one frame (`dt ≈ 0.0167s`) it would move roughly 4px right and 15px down if

unobstructed.

1. **X-pass**: move 4px right. Suppose this alone doesn't yet overlap the platform (the player is still above it) — no collision, `Velocity.X` unchanged.
2. **Y-pass**: move 15px down. Now the player's box overlaps the platform's top edge. `delta.Y < 0`, so this is a landing: push the player back up by `overlapY`, zero `Velocity.Y`, set `IsGrounded = true`.

Because the two axes were resolved independently, the outcome is unambiguous: the player ends the frame standing exactly on top of the platform, still carrying its horizontal speed. If both axes had been applied in one combined 4px-right-and-15px-down move before any check ran, and that combined move happened to land the box overlapping the platform from the side *and* the top simultaneously, resolving which single push-out direction is "correct" becomes genuinely ambiguous — which is precisely the class of bug (corner clipping, sticking to walls, tunneling) that axis separation avoids.

14. Glossary

AABB
Axis-Aligned Bounding Box — a rectangle (2D) or box (3D) whose edges are parallel to the coordinate axes; the simplest shape to test for overlap.

Clip space
The fixed -1..1 coordinate cube a vertex shader must ultimately output positions into; anything outside it is discarded before rasterization.

Coyote time
A short grace period after leaving solid ground during which a jump input is still honored, named after cartoon characters who don't fall until they look down.

deltaTime
The time, in seconds, elapsed since the previous frame; multiplying any per-second rate by it converts that rate into a distance/change for the current frame, keeping motion frame-rate independent.

EBO / IBO
Element (Index) Buffer Object — a buffer of integer indices into a vertex buffer, letting shapes reuse vertices instead of duplicating them.

Fragment shader
The GPU program that runs once per covered screen pixel and decides that pixel's final color.

Jump buffering
Remembering a jump key press for a short window so that pressing jump slightly before landing still triggers a jump on landing, instead of being dropped.

Mipmaps
Pre-shrunk copies of a texture at progressively lower resolutions, used so a texture drawn small on screen can sample from an appropriately-sized version instead of the full-resolution original.

NDC
Normalized Device Coordinates — another name for clip space after the perspective divide; in 2D orthographic rendering the two are effectively the same thing.

Orthographic projection
A projection with no perspective foreshortening — distance from the camera doesn't affect apparent size. Standard for 2D.

Shader program

A linked pair (here) of a compiled vertex shader and a compiled fragment shader, bound together via `UseProgram` before a draw call.

Texture unit

One of several GPU "slots" a texture can be bound into, referred to by index from a shader's `sampler2D` uniform.

Uniform

A shader input value set from C# that stays constant for an entire draw call, as opposed to vertex attribute data which varies per vertex.

VAO

Vertex Array Object — remembers which buffer is bound and how its data is laid out (the vertex attribute pointer configuration), so that setup doesn't need to be repeated before every draw call.

VBO

Vertex Buffer Object — a block of GPU memory holding raw vertex data.

Vertex shader

The GPU program that runs once per vertex and decides where that vertex lands on screen.

World space

The coordinate system a game's logic actually thinks in — here, ordinary pixel coordinates with the origin at the bottom-left — as opposed to the GPU's fixed clip space.

15. Current state & what's next

As of this document, the project has: a window that opens and clears to a color (Episode 1), a gradient triangle proving per-vertex color interpolation and per-pixel fragment shading (Episode 2/3), a real textured-quad sprite pipeline with a reusable `Shader` / `Texture` / `Sprite` stack (Episode 3), and — ahead of the original episode order — a playable platformer prototype with smooth acceleration/deceleration, gravity, coyote time, jump buffering, jump-cut, and axis-separated AABB collision against a small hand-built level (most of Episode 5, done early because it was asked for directly).

Not yet built, per the project's own roadmap (see `CLAUDE.md`):

- **Episode 4 — Tilemaps.** The current level is five hardcoded `Aabb` rectangles in `GameWindow.BuildLevel()`, not data loaded from an actual tile grid or level file.
- **Remaining Episode 5 polish** — slopes, moving platforms, and one-way platforms are not implemented; collision only handles static, solid rectangles.
- **Episode 6 — Polish.** No audio, particles, or on-screen text/UI yet.

This document reflects the code as it exists right now. As the project grows through these remaining episodes, regenerating it will keep it accurate — treat it as a snapshot tied to this point in the devlog, not a permanently up-to-date reference.